

Week2

Task1

To Apply Escape Sequences, Store Command History,
Navigate Previous Commands, Navigate Next Commands,
Update Input Buffer, Test Recall Functionality

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX_HISTORY 100
#define MAX_INPUT 1024

char history[MAX_HISTORY][MAX_INPUT];
int history_count = 0;
int history_pos = -1;

void add_history(const char *command) {
    if (strlen(command) == 0)
        return;
    if (history_count < MAX_HISTORY) {
        strcpy(history[history_count], command);
        history_count++;
    } else {
        for (int i = 1; i < MAX_HISTORY; i++)
            strcpy(history[i - 1], history[i]);
        strcpy(history[MAX_HISTORY - 1], command);
    }

    history_pos = history_count;
}

void show_history() {
    printf("\nCommand History:\n");
    for (int i = 0; i < history_count; i++)
        printf("%d %s\n", i + 1, history[i]);
}

void previous_command(char *buffer) {
    if (history_count == 0) {
        printf("\nNo command history.\n");
        return;
    }

    if (history_pos > 0)
        history_pos--;
    strcpy(buffer, history[history_pos]);
}
```

```

}

void next_command(char *buffer) {
    if (history_count == 0) {
        printf("\nNo command history.\n");
        return;
    }

    if (history_pos < history_count - 1) {
        history_pos++;
        strcpy(buffer, history[history_pos]);
    } else {
        history_pos = history_count;
        buffer[0] = '\0';
    }
}

int main() {
    char input[MAX_INPUT];

    printf("Simple Shell History Test\n");
    printf("Commands:\n");
    printf("  history      - Show command history\n");
    printf("  up           - Recall previous command\n");
    printf("  down         - Recall next command\n");
    printf("  exit         - Exit shell\n\n");

    while (1) {
        printf("myshell> ");

        if (fgets(input, MAX_INPUT, stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strcmp(input, "history") == 0) {
            show_history();
        }
    }
}

```

```

        if (strcmp(input, "history") == 0) {
            show_history();
            continue;
        }

        if (strcmp(input, "up") == 0) {
            previous_command(input);
            printf("Recalled: %s\n", input);
            continue;
        }

        if (strcmp(input, "down") == 0) {
            next_command(input);

            if (strlen(input) > 0)
                printf("Recalled: %s\n", input);
            else
                printf("No next command.\n");
            continue;
        }
        add_history(input);
        printf("Executed: %s\n", input);
    }
    printf("\nShell terminated.\n");
    return 0;
}

```

Output:

```

Commands:
  history      - Show command history
  up           - Recall previous command
  down         - Recall next command
  exit         - Exit shell

myshell> ls
Executed: ls
myshell> pwd
Executed: pwd
myshell> date
Executed: date
myshell> history

Command History:
1  ls
2  pwd
3  date
myshell> up
Recalled: date
myshell> down
No next command.
myshell> exit

Shell terminated.

```

Task2

To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    int *data;
    int size;
    int capacity;
} DynamicArray;

typedef struct Node {
    int data;
    struct Node *next;
} Node;

void initArray(DynamicArray *arr, int capacity) {
    arr->data = (int *)malloc(capacity * sizeof(int));
    arr->size = 0;
    arr->capacity = capacity;

    if (arr->data == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }
}

void resizeArray(DynamicArray *arr, int newCapacity) {
    int *temp = (int *)realloc(arr->data, newCapacity * sizeof(int));

    if (temp == NULL) {
        printf("Array resizing failed!\n");
        return;
    }

    arr->data = temp;
    arr->capacity = newCapacity;
}
```

```

void insertArray(DynamicArray *arr, int value) {
    if (arr->size == arr->capacity) {
        resizeArray(arr, arr->capacity * 2);
    }

    arr->data[arr->size++] = value;
}

void displayArray(DynamicArray *arr) {
    printf("Dynamic Array: ");

    for (int i = 0; i < arr->size; i++) {
        printf("%d ", arr->data[i]);
    }

    printf("\n");
}

Node *createNode(int value) {
    Node *newNode = (Node *)malloc(sizeof(Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = value;
    newNode->next = NULL;

    return newNode;
}

void insertList(Node **head, int value) {
    Node *newNode = createNode(value);

    newNode->next = *head;
    *head = newNode;
}

```

```
void displayList(Node *head) {
    printf("Linked List: ");

    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }

    printf("NULL\n");
}

void freeList(Node **head) {
    Node *temp;

    while (*head != NULL) {
        temp = *head;
        *head = (*head)->next;
        free(temp);
    }
}

int main() {
    DynamicArray arr;
    Node *head = NULL;

    initArray(&arr, 2);
    insertArray(&arr, 10);
    insertArray(&arr, 20);
    insertArray(&arr, 30);
    insertArray(&arr, 40);

    displayArray(&arr);

    insertList(&head, 100);
    insertList(&head, 200);
    insertList(&head, 300);

    displayList(head);
}
```

```

int main() {
    DynamicArray arr;
    Node *head = NULL;

    initArray(&arr, 2);
    insertArray(&arr, 10);
    insertArray(&arr, 20);
    insertArray(&arr, 30);
    insertArray(&arr, 40);

    displayArray(&arr);

    insertList(&head, 100);
    insertList(&head, 200);
    insertList(&head, 300);

    displayList(head);

    free(arr.data);
    freeList(&head);

    arr.data = NULL;
    head = NULL;

    printf("Memory released successfully.\n");

    return 0;
}

```

Output

```

Dynamic Array: 10 20 30 40
Linked List: 300 -> 200 -> 100 -> NULL
Memory released successfully.

```

Task3:

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_TOKENS 100

int main() {
    char input[MAX_INPUT];

    while (1) {
        printf("myShell> ");
        fgets(input, sizeof(input), stdin);

        input[strcspn(input, "\n")] = '\0';

        // Handle empty command
        if (strlen(input) == 0)
            continue;

        char *args[MAX_TOKENS];
        int count = 0;

        // Tokenization
        char *token = strtok(input, " \t");

        while (token != NULL && count < MAX_TOKENS - 1) {
            args[count++] = token;
            token = strtok(NULL, " \t");
        }

        args[count] = NULL;

        printf("--- Parsed Arguments ---\n");

        for (int i = 0; i < count; i++) {
            printf("Argument %d: %s\n", i + 1, args[i]);
        }
    }
}
```



```

        token = strtok(NULL, " \\t");
    }

    args[count] = NULL;

    printf("--- Parsed Arguments ---\n");

    for (int i = 0; i < count; i++) {
        printf("Argument %d: %s\n", i + 1, args[i]);
    }

    if (strcmp(args[0], "exit") == 0)
        break;

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        continue;
    }

    if (pid == 0) {
        execvp(args[0], args);

        perror("Command execution failed");
        exit(EXIT_FAILURE);
    }
    else {
        printf("Child PID: %d\n", pid);

        waitpid(pid, NULL, 0);

        printf("Command execution completed.\n");
    }
}

return 0;
}

```

Output:

```

root@kali:~/myShell# ./myShell
myShell> ls
--- Parsed Arguments ---
Argument 1: ls
Child PID: 1439
OS      bb.c.save  cd.c      co.c
OSSP    bc.c        child     code
OS_Lab  c.c.save  child.c  code.c
b.c.save cd         co       codes
Command execution completed.
myShell> exit
--- Parsed Arguments ---
Argument 1: exit

```

Task4:

To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

Source code:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define MAX 100

typedef struct {
    char command[MAX];
    char *args[MAX];
    int argc;
} Command;

void parseCommand(char *input, Command *cmd) {
    cmd->argc = 0;

    char *token = strtok(input, " \\t\\n");

    while (token != NULL && cmd->argc < MAX - 1) {
        cmd->args[cmd->argc++] = token;
        token = strtok(NULL, " \\t\\n");
    }

    cmd->args[cmd->argc] = NULL;

    if (cmd->argc > 0) {
        strcpy(cmd->command, cmd->args[0]);
    } else {
        cmd->command[0] = '\\0';
    }
}

int validateSyntax(Command *cmd) {
    if (cmd->argc == 0)
        return 0; // Empty command

    // Basic syntax validation
    for (int i = 0; i < cmd->argc; i++) {
        if (cmd->args[i][0] == '\\0')
            return -1;
    }
}
```

```

    return 1;
}

void printParseTree(Command *cmd) {
    printf("\nParse Tree:\n");
    printf("COMMAND\n");
    printf("  |\n");
    printf("  +-- %s\n", cmd->command);

    for (int i = 1; i < cmd->argc; i++) {
        printf("      +-- ARGUMENT: %s\n", cmd->args[i]);
    }
}

int main() {
    char input[MAX];
    Command cmd;

    printf("Enter command: ");
    fgets(input, sizeof(input), stdin);

    parseCommand(input, &cmd);

    // Handle empty command
    if (cmd.argc == 0) {
        printf("Empty command.\n");
        return 0;
    }

    // Validate syntax
    if (validateSyntax(&cmd) == -1) {
        printf("Syntax error detected.\n");
        return 1;
    }

    printf("\nCommand: %s\n", cmd.command);
    printf("Number of arguments: %d\n", cmd.argc - 1);

    printParseTree(&cmd);

```

```

    printParseTree(&cmd);

    printf("\nExecution structure:\n");
    for (int i = 0; i < cmd.argc; i++) {
        printf("args[%d] = %s\n", i, cmd.args[i]);
    }

    return 0;
}

```

Output:

Enter command: date

Command: date

Number of arguments: 0

Parse Tree:

COMMAND

|

+- date

Execution structure:

args[0] = date